In [1]:

```python
import itertools
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import NullFormatter
import pandas as pd
import numpy as np
import matplotlib.ticker as ticker
from sklearn import preprocessing
%matplotlib inline
```

In [2]:

```python
import pandas as pd
df = pd.read_csv('C:\\Users\\tanma\\Desktop\\ALL\\2nd same isi\\Machine learning assigment\\loan_train (1).csv')
df.head()
```

Out[2]:

	Unnamed: 0.1	Unnamed: 0	loan_status	Principal	terms	effective_date	due_date	age	education	Gender
0	0	0	PAIDOFF	1000	30	9/8/2016	10/7/2016	45	High School or Below	male
1	2	2	PAIDOFF	1000	30	9/8/2016	10/7/2016	33	Bechalor	female
2	3	3	PAIDOFF	1000	15	9/8/2016	9/22/2016	27	college	male
3	4	4	PAIDOFF	1000	30	9/9/2016	10/8/2016	28	college	female
4	6	6	PAIDOFF	1000	30	9/9/2016	10/8/2016	29	college	male

Dataset

This dataset contains historical loan data for 346 customers, capturing whether their loans were successfully paid off or ended up in default (collection). The dataset includes the following key features:

Loan_status: Indicates the loan outcome—Paid Off (repaid successfully) or Collection (defaulted).

Principal: The original loan amount borrowed by the customer.

Terms: The repayment schedule, which can be weekly (7 days), biweekly, or monthly.

Effective_date: The date when the loan was approved and became active.

Due_date: The final repayment due date (since this is a one-time payoff loan structure).

Age: The age of the applicant at the time of borrowing.

Education: The education level of the borrower.

Gender: The gender of the applicant (likely categories: Male, Female, or Other).

In [3]:

```python
df['due_date'] = pd.to_datetime(df['due_date'])
df['effective_date'] = pd.to_datetime(df['effective_date'])
df.head()
```

Out[3]:

	Unnamed: 0.1	Unnamed: 0	loan_status	Principal	terms	effective_date	due_date	age	education	Gender
0	0	0	PAIDOFF	1000	30	2016-09-08	2016-10-07	45	High School or Below	male
1	2	2	PAIDOFF	1000	30	2016-09-08	2016-10-07	33	Bechalor	female
2	3	3	PAIDOFF	1000	15	2016-09-08	2016-09-22	27	college	male
3	4	4	PAIDOFF	1000	30	2016-09-09	2016-10-08	28	college	female
4	6	6	PAIDOFF	1000	30	2016-09-09	2016-10-08	29	college	male

In [4]:

```python
df['loan_status'].value_counts()
```

Out[4]:

```
loan_status
PAIDOFF       260
COLLECTION     86
Name: count, dtype: int64
```

In [5]:

```python
# notice: installing seaborn might takes a few minutes
!conda install -c anaconda seaborn -y
```

```
Collecting package metadata (current_repodata.json): ...working... done
Solving environment: ...working... done

# All requested packages already installed.


==> WARNING: A newer version of conda exists. <==
  current version: 23.7.4
  latest version: 25.3.1

Please update conda by running

    $ conda update -n base -c defaults conda

Or to minimize the number of packages updated during conda update use

     conda install conda=25.3.1
```

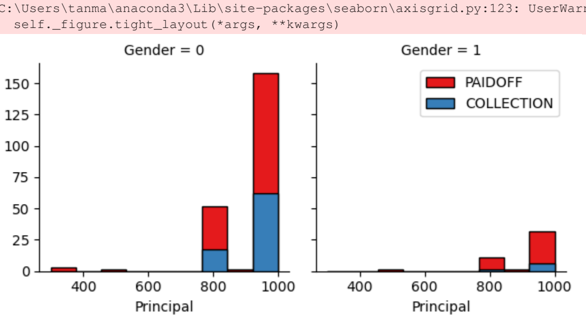
In [43]:

```python
import seaborn as sns

bins = np.linspace(df.Principal.min(), df.Principal.max(), 10)
g = sns.FacetGrid(df, col="Gender", hue="loan_status", palette="Set1", col_wrap=2)
g.map(plt.hist, 'Principal', bins=bins, ec="k")

g.axes[-1].legend()
plt.show()
```

```
C:\Users\tanma\anaconda3\Lib\site-packages\seaborn\axisgrid.py:123: UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```



Conclusion

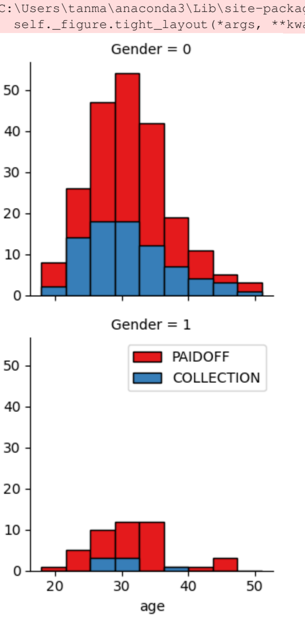
For both genders:Loans with lower principal amounts are more frequently paid off.As principal amount increases, the proportion of loans going into collection tends to increase.For Males:A broader spread of loan amounts is observed.More unpaid loans (collection status) appear in the higher principal ranges.For Females:Loan amounts might be slightly lower on average.Fewer instances of loans in collection, suggesting better repayment behavior in this sample.Loan repayment seems negatively associated with higher principal amounts.Gender differences in loan behavior may exist but should be confirmed with statistical testing.

In [39]:

```python
bins = np.linspace(df.age.min(), df.age.max(), 10)
g = sns.FacetGrid(df, col="Gender", hue="loan_status", palette="Set1", col_wrap=1)
g.map(plt.hist, 'age', bins=bins, ec="k")

g.axes[-1].legend()
plt.show()
```

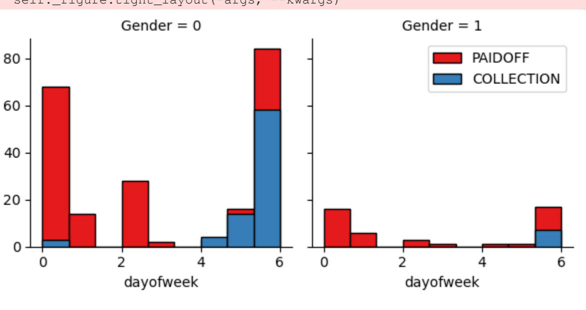
```
C:\Users\tanma\anaconda3\Lib\site-packages\seaborn\axisgrid.py:123: UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```



In [40]:

```python
df['dayofweek'] = df['effective_date'].dt.dayofweek
bins = np.linspace(df.dayofweek.min(), df.dayofweek.max(), 10)
g = sns.FacetGrid(df, col="Gender", hue="loan_status", palette="Set1", col_wrap=2)
g.map(plt.hist, 'dayofweek', bins=bins, ec="k")
g.axes[-1].legend()
plt.show()
```

```
C:\Users\tanma\anaconda3\Lib\site-packages\seaborn\axisgrid.py:123: UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```



In [41]:

```python
df['weekend'] = df['dayofweek'].apply(lambda x: 1 if (x>3)  else 0)
df.head()
```

Out[41]:

	Unnamed: 0.1	Unnamed: 0	loan_status	Principal	terms	effective_date	due_date	age	education	Gender	dayofweek	weekend
0	0	0	PAIDOFF	1000	30	2016-09-08	2016-10-07	45	High School or Below	0	3	0
1	2	2	PAIDOFF	1000	30	2016-09-08	2016-10-07	33	Bechalor	1	3	0
2	3	3	PAIDOFF	1000	15	2016-09-08	2016-09-22	27	college	0	3	0
3	4	4	PAIDOFF	1000	30	2016-09-09	2016-10-08	28	college	1	4	1
4	6	6	PAIDOFF	1000	30	2016-09-09	2016-10-08	29	college	0	4	1

In [42]:

```python
df.groupby(['Gender'])['loan_status'].value_counts(normalize=True)
```

Out[42]:

```
Gender  loan_status
0       PAIDOFF        0.731293
        COLLECTION     0.268707
1       PAIDOFF        0.865385
        COLLECTION     0.134615
Name: proportion, dtype: float64
```

In [11]:

```python
df['Gender'].replace(to_replace=['male','female'], value=[0,1],inplace=True)
df.head()
```

Out[11]:

	Unnamed: 0.1	Unnamed: 0	loan_status	Principal	terms	effective_date	due_date	age	education	Gender	dayofweek	weekend
0	0	0	PAIDOFF	1000	30	2016-09-08	2016-10-07	45	High School or Below	0	3	0
1	2	2	PAIDOFF	1000	30	2016-09-08	2016-10-07	33	Bechalor	1	3	0
2	3	3	PAIDOFF	1000	15	2016-09-08	2016-09-22	27	college	0	3	0
3	4	4	PAIDOFF	1000	30	2016-09-09	2016-10-08	28	college	1	4	1
4	6	6	PAIDOFF	1000	30	2016-09-09	2016-10-08	29	college	0	4	1

In [12]:

```python
df.groupby(['education'])['loan_status'].value_counts(normalize=True)
```

Out[12]:

```
education             loan_status
Bechalor              PAIDOFF        0.750000
                      COLLECTION     0.250000
High School or Below  PAIDOFF        0.741722
                      COLLECTION     0.258278
Master or Above       COLLECTION     0.500000
                      PAIDOFF        0.500000
college               PAIDOFF        0.765101
                      COLLECTION     0.234899
Name: proportion, dtype: float64
```

In [13]:

```python
df[['Principal','terms','age','Gender','education']].head()
```

Out[13]:

	Principal	terms	age	Gender	education
0	1000	30	45	0	High School or Below
1	1000	30	33	1	Bechalor
2	1000	15	27	0	college
3	1000	30	28	1	college
4	1000	30	29	0	college

In [14]:

```python
Feature = df[['Principal','terms','age','Gender','weekend']]
Feature = pd.concat([Feature,pd.get_dummies(df['education'])], axis=1)
Feature.drop(['Master or Above'], axis = 1,inplace=True)
Feature.head()
```

Out[14]:

	Principal	terms	age	Gender	weekend	Bechalor	High School or Below	college
0	1000	30	45	0	0	False	True	False
1	1000	30	33	1	0	True	False	False
2	1000	15	27	0	0	False	False	True
3	1000	30	28	1	1	False	False	True
4	1000	30	29	0	1	False	False	True

In [15]:

```python
X = Feature
X[0:5]
```

Out[15]:

	Principal	terms	age	Gender	weekend	Bechalor	High School or Below	college
0	1000	30	45	0	0	False	True	False
1	1000	30	33	1	0	True	False	False
2	1000	15	27	0	0	False	False	True
3	1000	30	28	1	1	False	False	True
4	1000	30	29	0	1	False	False	True

In [16]:

```python
y = df['loan_status'].values
y[0:5]
```

Out[16]:

```
array(['PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF'],
      dtype=object)
```

In [17]:

```python
X= preprocessing.StandardScaler().fit(X).transform(X)
X[0:5]
print(X.shape)
```

```
(346, 8)
```

In [18]:

```python
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
from sklearn import metrics
```

In [19]:

```python
train_X , test_X , train_y , test_y = train_test_split(X,y,test_size=0.2, random_state = 1)
```

In [20]:

```python
error_rate_test = []
KK = 12

for i in range(1,KK):

    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(train_X, train_y)
    pred_i = knn.predict(test_X)
    error_rate_test.append(np.mean(pred_i != test_y))



error_rate_train = []
for y in range(1,KK):

    knn = KNeighborsClassifier(n_neighbors=y)
    knn.fit(train_X, train_y)
    pred_i = knn.predict(train_X)
    #print(pred_i != train_y)
    error_rate_train.append(np.mean(pred_i != train_y))
    plt.figure(figsize=(10,6))
plt.plot(range(1,KK),error_rate_test, linestyle='-', marker='o',label="test",
         markerfacecolor='red', markersize=5)
plt.plot(range(1,KK),error_rate_train, linestyle='-', marker='o',label="train",
         markerfacecolor='blue', markersize=5)
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
plt.legend()

#model select k=7
KNN_model = KNeighborsClassifier(n_neighbors = 7)
KNN_model.fit(train_X, train_y)

#performance

accuracy1 = metrics.accuracy_score(train_y, KNN_model.predict(train_X))
print('Training acc: %.3f' % accuracy1)

accuracy2 = metrics.accuracy_score(test_y, KNN_model.predict(test_X))
print('Testing acc: %.3f' % accuracy2)
```

```
Training acc: 0.815
Testing acc: 0.757
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
<Figure size 1000x600 with 0 Axes>
```



Decision Tree

In [21]:

```python
from sklearn.tree import DecisionTreeClassifier
```

In [22]:

```python
DTC = DecisionTreeClassifier()
DTC_model = DTC.fit(train_X, train_y)
```

In [23]:

```python
#performance

accuracy1 = metrics.accuracy_score(train_y, DTC_model.predict(train_X))
print('Training acc: %.2f' % accuracy1)

accuracy2 = metrics.accuracy_score(test_y, DTC_model.predict(test_X))
print('Testing acc: %.2f' % accuracy2)
```

```
Training acc: 0.91
Testing acc: 0.69
```

Support Vector Machine

In [24]:

```python
import sklearn.svm as svm
```

In [25]:

```python
SVM_model = svm.SVC()
SVM_model.fit(train_X, train_y)
```

Out[25]:

```
▾ SVC
SVC()
```

In [26]:

```python
#performance

accuracy1 = metrics.accuracy_score(train_y, SVM_model.predict(train_X))
print('Training acc: %.2f' % accuracy1)

accuracy2 = metrics.accuracy_score(test_y, SVM_model.predict(test_X))
print('Testing acc: %.2f' % accuracy2)
```

```
Training acc: 0.76
Testing acc: 0.74
```

Logistic Regression

In [27]:

```python
from sklearn.linear_model import LogisticRegression
```

In [28]:

```python
logreg_model = LogisticRegression()
logreg_model.fit(train_X, train_y)
```

Out[28]:

```
▾ LogisticRegression
LogisticRegression()
```

In [29]:

```python
yhat = logreg_model.predict(test_X)
yhat
```

Out[29]:

```
array(['PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'COLLECTION',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'COLLECTION', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF',
       'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF', 'PAIDOFF'],
      dtype=object)
```

In [30]:

```python
#performance

accuracy1 = metrics.accuracy_score(train_y, logreg_model.predict(train_X))
print('Training acc: %.2f' % accuracy1)

accuracy2 = metrics.accuracy_score(test_y, logreg_model.predict(test_X))
print('Testing acc: %.2f' % accuracy2)
```

```
Training acc: 0.73
Testing acc: 0.77
```

Model Evaluation using Test set

In [31]:

```python
from sklearn.metrics import jaccard_score
from sklearn.metrics import f1_score
from sklearn.metrics import log_loss
```

In [32]:

```python
test_df = pd.read_csv('C:\\Users\\tanma\\Desktop\\ALL\\2nd same isi\\Machine learning assigment\\loan_train (1).csv')
test_df.head()
```

Out[32]:

	Unnamed: 0.1	Unnamed: 0	loan_status	Principal	terms	effective_date	due_date	age	education	Gender
0	0	0	PAIDOFF	1000	30	9/8/2016	10/7/2016	45	High School or Below	male
1	2	2	PAIDOFF	1000	30	9/8/2016	10/7/2016	33	Bechalor	female
2	3	3	PAIDOFF	1000	15	9/8/2016	9/22/2016	27	college	male
3	4	4	PAIDOFF	1000	30	9/9/2016	10/8/2016	28	college	female
4	6	6	PAIDOFF	1000	30	9/9/2016	10/8/2016	29	college	male

In [33]:

```python
test_df['effective_date'] = pd.to_datetime(test_df['effective_date'])
test_df['Gender'].replace(to_replace=['male','female'], value=[0,1],inplace=True)
test_df['dayofweek'] = test_df['effective_date'].dt.dayofweek
test_df['weekend'] = test_df['dayofweek'].apply(lambda x: 1 if (x>3)  else 0)
te_Feature = test_df[['Principal','terms','age','Gender','weekend']]
te_Feature = pd.concat([te_Feature,pd.get_dummies(test_df['education'])], axis=1)
te_Feature.drop(['Master or Above'], axis = 1,inplace=True)
te_Feature = preprocessing.StandardScaler().fit(te_Feature).transform(te_Feature)
```

In [34]:

```python
#KNN
KNN_acc = np.mean(KNN_model.predict(te_Feature) == test_df['loan_status'])
print('KNN acc: %.2f' %KNN_acc)
#DT
DT_acc = np.mean(DTC_model.predict(te_Feature) == test_df['loan_status'])
print('DT acc: %.2f' %DT_acc)
#SVM
SVM_acc = np.mean(SVM_model.predict(te_Feature) == test_df['loan_status'])
print('SVM acc: %.2f' %SVM_acc)
#Logreg
logreg_acc = np.mean(logreg_model.predict(te_Feature) == test_df['loan_status'])
print('logreg acc: %.2f' %DT_acc)
```

```
KNN acc: 0.80
DT acc: 0.86
SVM acc: 0.76
logreg acc: 0.86
```

In [35]:

```python
########################KNN
f1_KNN = f1_score(np.array(test_df['loan_status']), np.array(KNN_model.predict(te_Feature)), pos_label = 'PAIDOFF')
jaccard_KNN = jaccard_score(np.array(test_df['loan_status']), np.array(KNN_model.predict(te_Feature)), pos_label = 'PAIDOFF')

#######################Decision Tree
f1_DT = f1_score(np.array(test_df['loan_status']), np.array(DTC_model.predict(te_Feature)), pos_label = 'PAIDOFF')
jaccard_DT = jaccard_score(np.array(test_df['loan_status']), np.array(DTC_model.predict(te_Feature)), pos_label = 'PAIDOFF')

###############################################SVM
f1_SVM = f1_score(np.array(test_df['loan_status']), np.array(SVM_model.predict(te_Feature)), pos_label = 'PAIDOFF')
jaccard_SVM = jaccard_score(np.array(test_df['loan_status']), np.array(SVM_model.predict(te_Feature)), pos_label = 'PAIDOFF')

#########################################3Logreg
f1_logreg = f1_score(np.array(test_df['loan_status']), np.array(logreg_model.predict(te_Feature)), pos_label = 'PAIDOFF')
jaccard_logreg = jaccard_score(np.array(test_df['loan_status']), np.array(logreg_model.predict(te_Feature)), pos_label = 'PAIDOFF')


print(f'f1_KNN: {round(f1_KNN,3)}, jaccard_KNN: {round(jaccard_KNN,3)}')
print(f'f1_DT: {round(f1_DT,3)}, jaccard_DT: {round(jaccard_DT,3)}')
print(f'f1_SVM: {round(f1_SVM,3)}, jaccard_SVM: {round(jaccard_SVM,3)}')
print(f'f1_Logreg: {round(f1_logreg,3)}, jaccard_Logreg: {round(jaccard_logreg,3)}')
```

```
f1_KNN: 0.875, jaccard_KNN: 0.779
f1_DT: 0.906, jaccard_DT: 0.829
f1_SVM: 0.862, jaccard_SVM: 0.758
f1_Logreg: 0.849, jaccard_Logreg: 0.738
```

In [36]:

```python
#########################################LogLoss for logistic regression

logloss_logreg = log_loss(test_df['loan_status'],  logreg_model.predict_proba(te_Feature))


print(f'log loss: {round(logloss_logreg,3)}')
```

```
log loss: 0.435
```

In [37]:

```python
data = {
    'Algorithm': ['KNN', 'Decision Tree', 'SVM', 'LogisticRegression'],
    'Jaccard': [0.779,0.829,0.758,0.738],
    'F1-score': [0.875,0.906,0.862,0.849],
    'LogLoss': ['NA', 'NA', 'NA', 0.435]
}

report_df = pd.DataFrame(data)
print(report_df)
```

```
            Algorithm  Jaccard  F1-score LogLoss
0                 KNN    0.779     0.875      NA
1       Decision Tree    0.829     0.906      NA
2                 SVM    0.758     0.862      NA
3  LogisticRegression    0.738     0.849   0.435
```

In [ ]: